

Team Contract



TixHub — Event Ticket Sales Web Application

Introduction to Software Engineering (Intro2SE) — 24C11

Group 02 · School of Engineering

Established: June, 2026

Abstract — This document constitutes the formal working agreement among the five members of Group 02 for the duration of the TixHub project. It defines team roles and responsibilities, communication and meeting protocols, the work schedule and contingency plans, code and documentation standards, accountability and performance criteria, the decision-making and conflict resolution processes, and the procedure by which this contract is itself reviewed and amended. All members are bound by the terms herein upon signing.

Contents

1. [Team Roles and Responsibilities](#)
2. [Goals and Commitment Levels](#)

3. [Communication Plan](#)
 4. [Work Schedule and Deadlines](#)
 5. [Work Norms and Task Management](#)
 6. [Code and Documentation Standards](#)
 7. [Accountability and Performance](#)
 8. [Decision-Making and Conflict Resolution](#)
 9. [Review and Update Process](#)
 10. [Commitment and Signatures](#)
-

1. Team Roles and Responsibilities

Name	Student ID	Role	Primary Responsibilities	Note
Lương Hưng Phát	24127298	PM / Backend Engineer	Project management, sprint planning, backend development, final sprint sign-off	Team Leader
Nguyễn Thành Đạt	24127021	Fullstack Dev / Tester	Cross-cutting features, QA, testing	
Nguyễn Minh Khoa	24127188	Backend Developer	Feature development, backend modules	
Nguyễn Tấn Hiệu	24127373	Frontend Dev / DevOps	UI/UX implementation, frontend features, CI/CD, hosting infrastructure	
Nguyễn Anh Khôi	24127430	DB Management	Schema design, migrations, data integrity	

1.1 Full-Stack Expectation

The roles above set *expectations* for ownership and leadership of a domain, they do **not** restrict a member to that domain. The role owner is expected to lead their phase (e.g., the Frontend/DevOps owner leads UI/UX and CI/CD decisions), while all other members are expected to participate in, and be able to contribute to, every part of the system.

1.2 Task Assignment Model

Task assignment follows a **hybrid model**: roles define domain ownership, but sprint tasks are assigned dynamically each sprint based on capacity and priority.

2. Goals and Commitment Levels

2.1 Team Goal

The team's collective objective is to attain a good grade in the Intro2SE course. Given that the course places primary emphasis on development process and collaborative practices, specifically through adherence to Scrum methodology, the team recognizes that process quality plays an impactful role. The delivered product is expected to be functionally stable, free of critical defects, and accompanied by a well designed user interface. Should resource constraints arise, reductions in **feature scope** are considered acceptable; reductions in **work quality** are not.

2.2 Personal Goals

Each member individually commits to sustaining an A-grade level of effort throughout the project. In the event that a member determines that a lower grade outcome is acceptable to them personally, that member is obligated to communicate this change in commitment to the team without delay. **Silent underperformance** whereby a member reduces their contribution without disclosure is a violation of this contract.

2.3 Anticipated Obstacles

- Schedule conflicts between members
- Late task deliverables blocking dependent work
- Disagreements in work process and how criticism is given / received

2.4 Workload Distribution

Unequal workload distribution is **not acceptable** as a permanent state.

- Short-term cover (within 1 sprint) is permitted when a member is overwhelmed.
 - The covering member must flag it to the team lead immediately.
 - The member who was covered must take responsibility to compensate in the next sprint.
 - If a member is busy or blocked, they **must proactively inform the team lead** — silence is not an option.
-

3. Communication Plan

3.1 Tools

Purpose	Tool
Real-time chat / async standup	Discord (team channel)
Voice / video sync meetings	Discord / Google Meet
Task tracking	Jira
Source control & code review	GitHub (Pull Requests)
Document & minutes sharing	Shared team drive / channel

3.2 Recurring Meetings

Meeting	Frequency	Format
Sync meeting	2× per week	Online (Discord / Google Meet)
Daily standup	Daily	Async in team chat channel
Sprint planning	Start of each sprint	Online sync
Sprint retrospective	End of each sprint	Online sync

3.3 Response-Time Expectations

- Routine messages in the team channel: acknowledged within **12 hours** during the working week.
- Messages explicitly tagged **@urgent** (or direct mention on a blocker): acknowledged within **3 hours**.
- A member who will be unreachable for more than **24 hours** must notify the team in advance.
- "Acknowledged" means a reply, not necessarily a full resolution, but it must state when a full response will follow.

3.4 Meeting Records

- One **fixed note taker** is assigned for the duration of the project.
 - The note taker writes a meeting summary (minutes) for every meeting.
 - Minutes must be posted to the shared team channel **within 24 hours** of the meeting.
 - Minutes must include a list of **action items**, each with:
 - Owner (one person + one supporter)
 - Due date
-

4. Work Schedule and Deadlines

4.1 Milestones

The project runs ~3 months across **5 sprints**. Concrete sprint scope and deadlines are set

during each Sprint Planning session and recorded in Jira. Sprint boundaries are the binding

milestones; the final sprint includes a PM sign-off before submission.

4.2 Availability

- Members keep their general availability for meetings and work sessions current in the team channel.
- The two weekly sync meetings are mandatory; conflicts must be raised at least **24 hours** beforehand so the slot can be moved.

4.3 Rules for Missed Deadlines / Delays

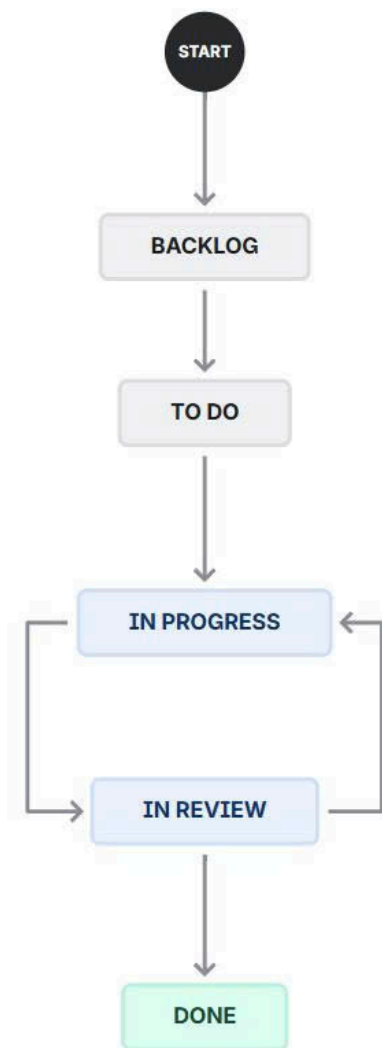
- A foreseeable delay must be flagged **before** the deadline, not after.
- Short term cover rules in Section 2.4 apply; repeated misses are handled under Section 7.3.
- Deadline changes require **mutual team agreement**.

5. Work Norms and Task Management

5.1 Task Management (Jira)

- All tasks are tracked in **Jira**.
- Every task has exactly **one owner** (accountable) and **one supporter** (secondary helper).
- Deadlines are set during sprint planning and are binding.

5.2 Jira Workflow



- Owner moves task to **In Review** when work is done.

- **Supporter + PM** validate the task before it moves to **Complete**.
 - All code changes require a **PR with at least 1 reviewer** (preferably the assigned supporter) before merge.
-

6. Code and Documentation Standards

6.1 Coding Conventions

- Follow the agreed language/framework style guide for the stack; formatting is enforced automatically (linter / formatter) so style is never a review topic.
- Branch naming and commit messages follow a consistent convention agreed at project start.
- No commented out dead code or debug prints in merged branches.

6.2 Code Review

- Every change reaches `main` through a **Pull Request** with **at least one reviewer** (preferably the task's supporter).
- Reviews are work focused and follow the criticism standard in Section 8.3.
- A PR is not merged while CI checks are failing.

6.3 Testing

- New features ship with tests appropriate to the change; bug fixes include a test that reproduces the bug.
- The Tester owns overall QA and regression coverage, but **authoring tests is every member's responsibility**, not the Tester's alone.
- The build and test suite must pass before a task moves to **Complete**.

6.4 Documentation

- Each significant feature/module carries enough documentation (README section or in-repo doc) for another member to run and extend it.
 - API contracts and database schema changes are documented at the time of the change.
 - Meeting minutes, decisions, and this contract live in the shared team space and are kept current.
-

7. Accountability and Performance

7.1 Contribution Criteria

Contribution is measured by **completed, reviewed, and accepted work** (tasks moved to Complete per Section 5.2), not by raw activity or hours claimed. Quality is measured against the standards in Section 6.

7.2 Handling Late Deliverables

Occurrence	Action
1st miss	Owner explains reason in team channel; task flagged
2nd miss	Team lead conducts a 1-on-1 conversation with the member
3rd miss	Escalate to TA / course instructor

7.3 Underperformance and Consequences

- Persistent failure to meet the commitment level in Section 2, after the steps in Section 7.2, is recorded and reflected in the peer/contribution assessment submitted to the course.

- Violations of this contract (e.g., silent underperformance, unilateral deadline changes) are raised to the whole team and, if unresolved, escalated to the TA / instructor.
-

8. Decision-Making and Conflict Resolution

8.1 Decision-Making Process

Decision Type	Examples	Process
Minor	Task approach, UI detail, naming choice	Majority vote — 3 of 5 members
Major	Architecture change, scope change, deadline shift, tool change	Full consensus — all 5 members must agree

The **team lead has the final say** only where a process is explicitly deadlocked (Section 8.2).

8.2 Resolving Quality Disagreements

1. Both sides present their position with evidence.
2. A maximum of **2 rounds** of discussion.
3. If still deadlocked after 2 rounds → **the team lead makes the final call**.
4. The decision is binding — no re-litigating after the meeting.

8.3 Criticism and Feedback Standards

Feedback must be:

- **Direct and structured** — vague criticism ("this is bad") is not acceptable.
- **Work-focused, not person-focused** — criticize the output, never the individual.
- **Evidence-based** — identify specifically what is wrong and why.
- **Actionable** — every critique must include a clear suggestion for improvement.

Unacceptable: "Your code is a mess."

Acceptable: "The authentication handler doesn't handle expired tokens — it will crash on

line 42. Suggest adding a try/catch with a 401 response."

8.4 Escalation

Disputes that cannot be resolved internally through Sections 8.1–8.2 are escalated to the

TA or course instructor. Escalation is a last resort, used only after the internal steps have been attempted and documented in the meeting minutes.

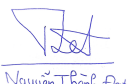
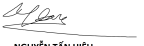
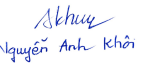
9. Review and Update Process

- This contract is reviewed at each **Sprint Retrospective** to confirm it still matches how the team actually works.
 - Any member may propose an amendment at any time via the team channel.
 - Amendments are adopted as a **Major decision** (full consensus, Section 8.1).
 - On adoption, the version number and date in the title block are incremented and the change is noted in the minutes. The contract is intentionally flexible and expected to evolve as the project progresses.
-

10. Commitment and Signatures

By signing below, each team member confirms they have read, understood, and agree to abide by

this contract for the duration of the TixHub project.

Name	Student ID	Role	Signature	Date
Lương Hưng Phát	24127298	PM / Backend Engineer	 Lương Hưng Phát	03/06/2026
Nguyễn Thành Đạt	24127021	Fullstack Dev / Tester	 Nguyễn Thành Đạt	03/06/2026
Nguyễn Minh Khoa	24127188	Backend Developer		03/06/2026
Nguyễn Tấn Hiệu	24127373	Frontend Dev / DevOps	 NGUYỄN TẤN HIỆU	03/06/2026
Nguyễn Anh Khôi	24127430	DB Management	 Nguyễn Anh Khôi	03/06/2026